AN IMPLEMENTED COMPILER / AN UNDERGRADUATE LABORATORY

Direct LLVM Implementation

Functions, objects, closures, and PicoCalc

Architecture and runtime contracts

Working source, native execution, and LLVM JIT

Recorded tests, engineering diary, and primary references

TWO VALIDATED EXECUTION TARGETS

Dynamic JavaScript: Linux x86-64 + Static Hermes runtime

Integer worlds: RV32IM code + PicoCalc instruction simulator

28 OCTOBER 2026 | EXPERIMENTAL BACKEND RELEASE

Hermes base: f837b9dd8396ebf5c4e8115412417d273565f0fe

Contents

1. What was implemented 3

2. Architecture and implementation 6

3. Validation, findings, and boundaries 16

4. PicoCalc integration and the device boundary 20

5. Build, inspect, execute, and reproduce 23

6. References and source-reading guide 26

This report explains the delivered implementation and the evidence that supports it. The accompanying source archive includes the complete source trees, a patch, exercises, raw logs and the detailed diary. Chapter 2 is the teaching core; Chapter 3 is the release evidence; Chapter 4 defines the remaining device integration boundary.

How to study the implementation

You should be comfortable with functions, pointers and basic data structures. Chapter 2 introduces intermediate representations, basic blocks, phi values, tagged values, roots, closure environments and ABI contracts. Its five exercises connect source programs to observable behavior. Chapter 5 gives the build and execution commands; Chapter 6 provides primary references and a guide to the source.

Read the final evidence separately from the engineering diary. The diary preserves decisions, experiments, failures and fixes. Earlier failed attempts explain the work; only the identified final runs support the reported validation counts.

1. What was implemented

1.1 The result

This release adds a working direct LLVM backend to Static Hermes and updates the earlier PicoCalc integer-world compiler to emit LLVM IR directly. The larger backend accepts real JavaScript functions, objects, escaping lambdas, mutable captured bindings, ordinary constructors and classes, and exception control flow. It has been compiled and executed against the actual Hermes runtime on a Linux x86-64 host, including execution through LLVM's `lli` JIT. The smaller PicoCalc backend generates RV32IM code and preserves the existing simulated world's transaction and generation protocol.

The distinction between these two targets is essential. The host backend supplies the dynamic language mechanisms requested in this update. The device-oriented prototype supplies a bounded integer ABI and an RV32 execution environment. They are both implemented, but the host's managed JavaScript heap has not been ported into the device world. No physical PicoCalc or ESP32-P4 execution is claimed.

The source baseline is Facebook's Static Hermes branch at commit `f837b9dd8396ebf5c4e8115412417d273565f0fe`. The accompanying patch adds the direct emitter and its integration; the pinned upstream commit alone does not contain these changes. The release includes a complete source snapshot, a patch that applies to that exact base, test fixtures, raw validation records, this report's Markdown source, and a detailed engineering diary.

1.2 What "direct to LLVM" means

The previous Static Hermes path translated Hermes IR into C, then relied on a C compiler to translate that program into LLVM IR and machine code. The new path translates Hermes IR into LLVM IR itself. The JavaScript frontend, semantic analysis, and useful module lowering remain; the program-specific C representation and C parser stage are removed.

A fixed C bridge remains as part of the runtime implementation. It is compiled once for a particular runtime configuration. This bridge implements stable entry points for the tagged value representation, root frames, environments, object operations, calls, and compilation units. Calling such a prebuilt runtime from generated LLVM is the same architectural pattern used when native code calls a standard library. It does not regenerate a C translation of each guest program.

The first implementation emits textual `.ll` files. LLVM still parses those files, verifies their structure, optimizes them, and emits native code. This choice makes the translation inspectable in a teaching laboratory and avoids linking the compiler frontend to LLVM's C++ libraries. An in-memory LLVM API or bitcode writer could remove the textual IR parsing step later, after the runtime contracts are stable. The current implementation makes no measured compilation-speed or execution-speed claim.

1.3 Feature map

The table names mechanisms exercised by the delivered tests. It is a feature inventory for this release, not a full ECMAScript conformance statement. Chapter 3 explains exactly how the evidence was collected.

Area	Implemented and exercised examples	Source to inspect
Functions	Named functions, recursive factorial, mutual recursion, higher-order and indirect calls	`functions.js`; `emitCall`; `shllvm_call`
Lambdas and closures	Arrows and function expressions; escaping, nested, recursive and shared mutable captures	`closures.js`; environment helpers
Lexical bindings	Per-iteration loop capture; closure-mediated temporal dead zones	`lexical_tdz.js`; compiler-default checks
Objects	Literals, computed keys, own-property definition, identity, cycles, getters and setters	`objects.js`; `accessors.js`
Arrays	Literal holes, indexed mutation, and callbacks into generated functions	`arrays_callbacks.js`
Invocation rules	`this`, lexical arrow receiver, arguments, constructors, `new.target` and rejected arrow construction	`arrows_this.js`; `arguments.js`; `constructors.js`
Classes	Ordinary inheritance, `super`, getters, captured receiver, `instanceof` and static inheritance	`classes.js`
Control and exceptions	Branches, loops, phi transfers, nested catch/finally, return and loop control through finally	`control_phi.js`; `exceptions.js`
Dynamic values	Strings, UTF-16 data, coercion, arithmetic and runtime error classes	`strings.js`; `coercion.js`; error fixtures
Collection	Escaping closures retain live state after allocation churn and explicit collection	`gc_closures.js`
Device world	Direct LLVM integer code, capability ecalls, bounded RV32 execution, transactional reload	PicoCalc compiler and supervisor tests

All host fixture names in this table are relative to `hermes/test/LLVMNative/` in the complete release.

1.4 A complete worked program

The runnable example `hermes/examples/llvm/counter.js` combines the requested features. A counter object contains three lambdas. The lambdas share a mutable binding; a second counter receives a different binding. A higher-order function produces an array callback. A class returns an arrow that keeps its receiver, and an exception object carries another retained function.

```
// A native-compiled closure shares a mutable environment with its siblings.
function makeCounter(initial) {
  let value = initial;
  return {
    add: delta => { value += delta; return value; },
    read: () => value,
    snapshot: () => ({ value, description: "captured state" })
  };
}

const first = makeCounter(40);
const second = makeCounter(100);
print(first.add(1), first.add(1), first.read(), second.add(5));
print(JSON.stringify(first.snapshot()));

// A function can return a function, and ordinary array methods call it back.
function multiplyBy(scale) {
  return value => value * scale;
}
print([1, 2, 3].map(multiplyBy(7)).join(","));

// Constructors, inheritance, methods and lexical arrow this use the same
// runtime calling convention as plain source functions.
class CounterView {
  constructor(counter) { this.counter = counter; }
  reader() { return () => this.counter.read(); }
}
const readLater = new CounterView(first).reader();
first.add(8);
print(readLater());

// Exception edges preserve committed captured state.
try {
  first.add(2);
  throw { reason: "example", read: first.read };
} catch (error) {
  print(error.reason, error.read());
}
```

The native executable and the `lli` JIT produced the same output:

```
41 42 42 105
{"value":42,"description":"captured state"}
7,14,21
50
example 52
```

The first line is a compact closure test. Both updates and the read through `first` reach the same environment, giving 41, 42, and 42. The other counter's update produces 105 independently. The snapshot creates a new ordinary object containing the current value. The array result demonstrates a returned function used as a callback. The value 50 confirms that the method's returned arrow can still find the counter after the method returns. Finally, the caught object retains a read closure and observes the later value 52.

The output is recorded evidence, not merely an expected result inferred from the example. Its native and JIT logs are `docs/llvm/evidence/counter-native.txt` and `counter-jit.txt`. The corresponding generated modules and build manifests can be reproduced with the driver in Chapter 5.

2. Architecture and implementation

2.1. What this implementation builds

This chapter follows a JavaScript program from Hermes's intermediate representation to LLVM IR and then to native execution. Its central question is practical: **how can a compiler emit native code for functions, objects, and closures while preserving the language's runtime behavior?** Arithmetic instructions alone are insufficient. A function can return another function, a property access can call a getter, and an allocation can move objects that an earlier expression still needs.

The implementation answers that question by combining a new LLVM emitter with the existing Static Hermes runtime. The emitter generates each program's functions, branches, and calls directly as textual LLVM IR. A small, separately compiled C bridge adapts those calls to Hermes's value representation, object model, environments, call frames, and garbage collector. The bridge is built once for a runtime configuration; a guest program does not become a generated C program on this path. [1] [2] [3]

The source base is the `static_h` development line of Hermes, pinned here to commit `f837b9dd8396ebf5c4e8115412417d273565f0fe`. This chapter describes the accompanying local changes, which are not present in that upstream commit. It explains implemented mechanisms and their constraints; examples are teaching examples, not a claim that every JavaScript feature or every optimization combination has passed validation. Consult the delivered validation records for measured results.

The current dynamic runtime bridge deliberately requires a 64-bit host pointer ABI. The separate PicoCalc prototype compiles its restricted integer worlds directly to RV32 LLVM IR. These are two distinct execution targets. Objects and closures in the host backend do not establish that the dynamic Hermes runtime runs on a physical PicoCalc or ESP32 device. Section 2.13 explains the remaining integration work.

2.2. Where the compiler changes direction

An **intermediate representation**, or IR, records program operations in a form designed for analysis and transformation. Hermes already parses JavaScript, resolves bindings, constructs functions and basic blocks, and lowers many language constructs. The new backend reuses that work. It does not parse JavaScript again.

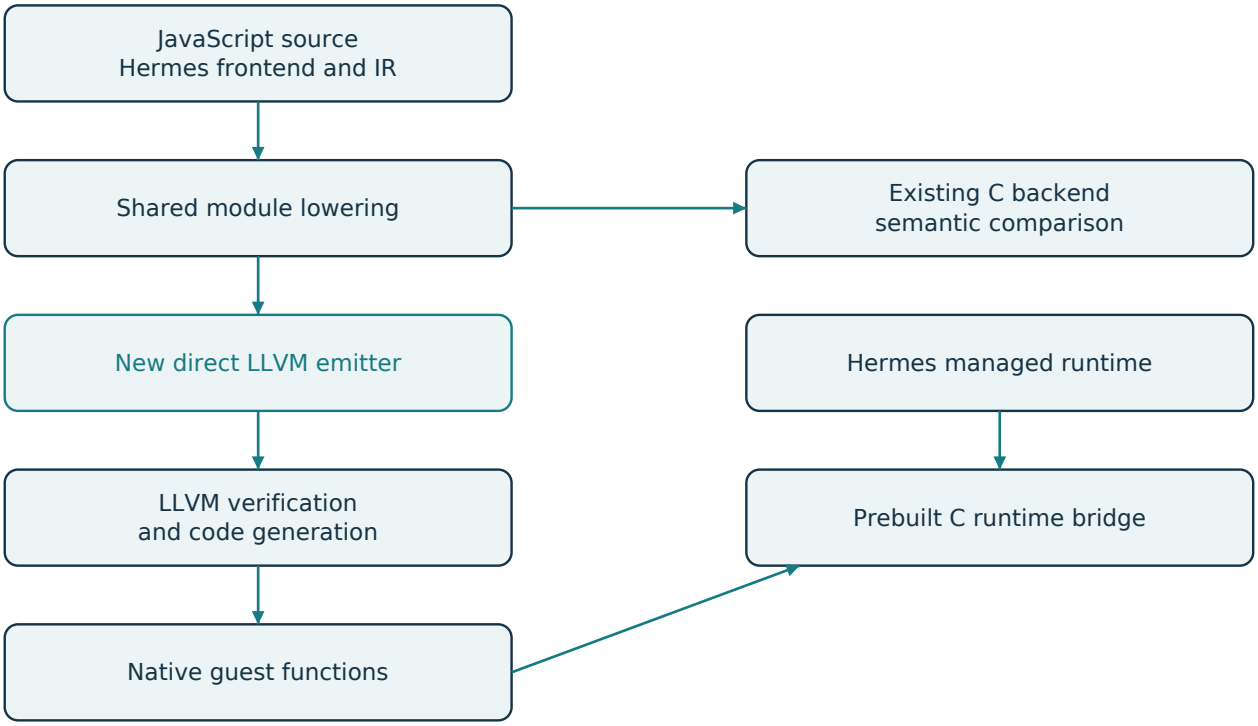


Figure 1. The direct emitter branches before C-specific register allocation. Generated functions call the prebuilt bridge; the existing C backend remains an independent comparison path.

The compiler driver's `-emit-llvm` path calls `sh::generateLLVM`. That function invokes `lowerForLLVM`, assigns indices to environment variables, and creates a `ModuleEmitter`. The shared lowering includes argument handling, object literal lowering, scope lowering, switch lowering, constant loading, and selected optimizations. [1] [4]

The boundary matters. The existing C backend subsequently performs its own register allocation and transformations such as call lowering that encode its register choices. The direct backend stops before those stages. LLVM receives a representation from which it can perform its own optimization and machine register allocation. Reusing the C backend's final register-oriented form would unnecessarily couple the new emitter to a different code generator's storage decisions.

Component	Responsibility
`tools/shermes/compile.cpp`	Select direct emission and publish a complete module only after backend success
`lib/BCGen/SH/SH.cpp`	Provide the shared module-lowering boundary
`LLVMNative/LLVMBackend.cpp`	Translate supported Hermes instructions, functions, control flow, and module data into LLVM text
`LLVMNative/runtime.h`	Declare the narrow interface between generated code and runtime adaptation
`LLVMNative/runtime.c`	Implement that interface using the actual Static Hermes runtime
`utils/shllvm.py`	Assemble, verify, optimize, link or JIT, and record the build inputs

`ModuleEmitter` assigns function IDs, interns strings, gathers helper declarations, and writes module data. `FunctionEmitter` handles an individual function's blocks, storage, instructions, and exception regions. Imported helper signatures are recorded consistently: a helper cannot silently acquire different LLVM argument types at different call sites. [1]

Text emission keeps the compiler independent of LLVM's C++ library ABI. The tools `llvm-as` and `opt` subsequently check the emitted module. Unsupported Hermes instructions produce diagnostics, and `compileToLLVM` buffers the module before publishing it. This prevents an incomplete translation from looking like successful output. Verification remains a structural check: valid LLVM can still implement the wrong JavaScript behavior.

2.3. Values and the runtime boundary

Generated native JavaScript functions currently have this shape:

```
define internal i64 @shllvm_f7(ptr %shr) {
  ; Generated function body.
}
```

The parameter is a runtime pointer. The returned `i64` holds the bits of a Hermes `SHLegacyValue`. It does **not** mean that every JavaScript value is an integer. Numbers, booleans, undefined, null, and heap references use the runtime's tagged representation. JavaScript arguments arrive through Hermes's register stack, described in Section 2.6. [1] [2] [5]

Most helpers receive pointers to rooted value slots and place their result into another slot. For example, a dynamic addition passes a result address, runtime pointer, operation code, and two operand addresses to `shllvm_binary`. The bridge then calls Hermes's existing addition implementation. This preserves behaviors such as string concatenation and object-to-primitive conversion without duplicating them in the emitter.

The LLVM-visible interface uses a small vocabulary:

LLVM type	Meaning in this interface
`ptr`	An opaque runtime, frame, unit, native function, or rooted-slot address, according to the helper signature
`i64`	Tagged value bits, or an explicitly declared size/bit-pattern field
`i32`	Counts, indices, operation selectors, and bridge boolean results
`double`	A Number already known to be suitable for floating-point arithmetic

Opaque pointers do not remove type discipline. Passing a frame address where a rooted value address is expected is still a compiler bug. The discipline resides in the helper prototypes and emitter logic rather than LLVM pointee types.

The bridge computes native structure sizes and accesses C fields. Compile-time assertions require eight-byte values, eight-byte host pointers, and a sixteen-byte function descriptor. These assertions document the present ABI contract; they are not a proof of portability to every 64-bit platform.

Constants also cross this boundary deliberately. Numeric bit patterns become values through `_sh_ljs_untrusted_double`; booleans use the boolean constructor. The internal Empty sentinel has its own representation. Uninit follows the existing SH convention of using undefined bits, while compiler type information retains its distinct role. Consequently, a temporal-dead-zone check carries an explicit invalid-type mask: it must not reject every ordinary JavaScript `undefined`. [1] [2] [5]

2.4. Keeping values alive across allocation

A **garbage-collector root** is a location from which the collector begins tracing live objects. A native machine register containing an object address is not automatically such a location. Moreover, a moving collector can relocate an object and update its registered references. Generated code must subsequently use the updated value.

This backend adopts a conservative storage policy: every Hermes instruction receives a root slot, and encountered constants receive slots too. Side-effect-only instructions also receive unused slots to keep numbering simple. A scratch slot supports temporary helper operations. All slots are initialized to undefined before they are exposed to collection. [1] [2]

At entry, generated code asks `shllvm_frame_size` for the required native storage size, allocates aligned stack storage, and calls `shllvm_enter`. The bridge initializes the `SHLocals` header, links this root frame through `_sh_enter`, and reserves the outgoing Hermes argument/frame area. The emitter obtains the root-array address through `shllvm_roots`; it never guesses the offset of that array inside `SHLocals`.

The resulting invariant is:

> Every guest value that must survive a potentially allocating operation has an authoritative copy in a location traced by Hermes.

A property read can invoke a getter; a conversion can invoke user code; either can allocate. Therefore the emitter treats ordinary runtime helpers as possible safepoints unless their implementation establishes otherwise. After a helper returns, later uses reload their values from root slots. The emitter does not cache a movable heap address in a long-lived LLVM temporary. It also avoids false purity attributes that would tell LLVM these calls cannot modify rooted storage.

Short-lived raw values are permissible between safepoints. A phi-edge transfer, for example, performs only loads and stores. Similarly, argument copying first loads rooted values into an outgoing array, then the bridge copies them without allocation into Hermes's traced register stack before invoking the callee. The temporary array is not a separate collector root table; the copying protocol makes that safe. [1] [2] [6]

Normal return reads the result, removes any function-level exception handler, leaves the runtime frame, and returns its bits. Those cleanup operations do not allocate managed objects. Exceptional unwinding needs a different restoration path, explained in Section 2.10.

This design favors inspectable correctness over compact frames. Values can remain rooted after their last semantic use, delaying collection until return. Precise liveness, slot reuse, or LLVM stack maps would be later optimizations requiring new proofs. The current admission limit is 1,024 combined root and argument slots, including scratch/minimum argument storage: at most 8 KiB of explicit tagged payload. Headers, exception buffers, machine spills, runtime frames, and recursion consume additional stack. The limit and native stack checks are not a total memory budget.

2.5. Branches and simultaneous phi assignments

A **basic block** contains instructions executed in sequence until its final branch, return, or throw. A conditional Hermes branch becomes a truthiness helper call followed by an LLVM conditional branch. JavaScript truthiness is preserved before LLVM receives its one-bit condition. [1]

Hermes IR also uses phi nodes. A phi selects a value according to the incoming control-flow edge. In this implementation its result lives in a root slot. The emitter inserts an edge block whenever the destination requires phi transfers.

Consider a loop whose next iteration exchanges two values:

```
next_a = old_b
next_b = old_a
```

Writing `a = b` and then `b = a` into the same storage loses `old_a`. Both phi assignments logically occur together. `emitEdges` therefore loads **all incoming values first**, then stores all destination slots:

```
edge_example:
  %old_a = load i64, ptr %a, align 8
  %old_b = load i64, ptr %b, align 8
  store i64 %old_b, ptr %a, align 8
  store i64 %old_a, ptr %b, align 8
  br label %loop
```

This is schematic explanatory IR, not an extracted module. There are no allocating calls between these transfers. Thus the raw temporaries remain valid, and cycles among phi assignments cannot overwrite their own inputs. The same edge mechanism handles phi transfers into exception destinations. LLVM's phi definition supplies the general control-flow semantics; the rooted-slot strategy is this backend's implementation choice. [1] [9]

2.6. Calling functions, passing `this`, and constructing objects

Although a generated function takes one LLVM parameter, a JavaScript call has a callee, arguments, `this`, and sometimes `new.target`. `emitCall` gathers explicit arguments into a contiguous array. `shllvm_call` places the callee, receiver, construction target, and arguments into the outgoing Hermes frame, then invokes `_sh_ljs_call`. Hermes owns the remaining call metadata and dispatch. [1] [2] [6]

`shllvm_param` reads incoming parameters using the runtime's established numbering: index zero is `this`, and explicit parameters begin at one. Missing parameters produce undefined. Recursion creates new native/root frames, while each invoked closure still identifies its own captured environment.

This distinction explains method calls. In `object.method(3)`, the function value and receiver are different pieces of call information. Extracting the same method into a variable can change the receiver on a later call. The frontend represents this distinction, and the bridge preserves it. Non-strict receiver coercion has its own helper; an arrow's lexical `this` comes from the frontend's capture representation, rather than a special rule invented by the call bridge.

Construction also uses the existing protocol. Helpers implement `CreateThis`, `GetNewTarget`, and selection between the allocated receiver and an explicit object returned by the constructor. Function metadata distinguishes permitted invocation forms: an arrow cannot be constructed, and a class constructor cannot be called as an ordinary function. The generated prologue checks these restrictions.

Class creation delegates to `_sh_ljs_create_class`, passing the native function, its environment and metadata, the home-object output, and an optional superclass. Receiver-sensitive property helpers support operations whose lookup object differs from their receiver. These mechanisms describe the implemented path; they do not imply that every class extension, such as private-field lowering, is accepted. Argument objects likewise retain Hermes's lazy/reified and strict/loose helper paths. [1] [2] [5]

2.7. Closures are code plus shared bindings

A **closure** packages executable code with the lexical environment needed by that code. A captured binding is a mutable cell in an environment, not necessarily a copy of the value present when the function was created.

Consider this excerpt from `test/LLVMNative/closures.js`:

```
function counter(start) {
  let n = start;
  return {
    inc: () => ++n,
    dec: function () { return --n; },
    read: () => n
  };
}
const a = counter(40), b = counter(100);
print(a.inc(), a.inc(), a.dec(), a.read(), b.inc(), a.read());
```

The expected output, derived from the source, is `41 42 41 41 101 41`. `a.inc`, `a.dec`, and `a.read` share the same binding for `n`. The second invocation creates a separate binding for `b`. Incrementing `b` cannot change `a`.

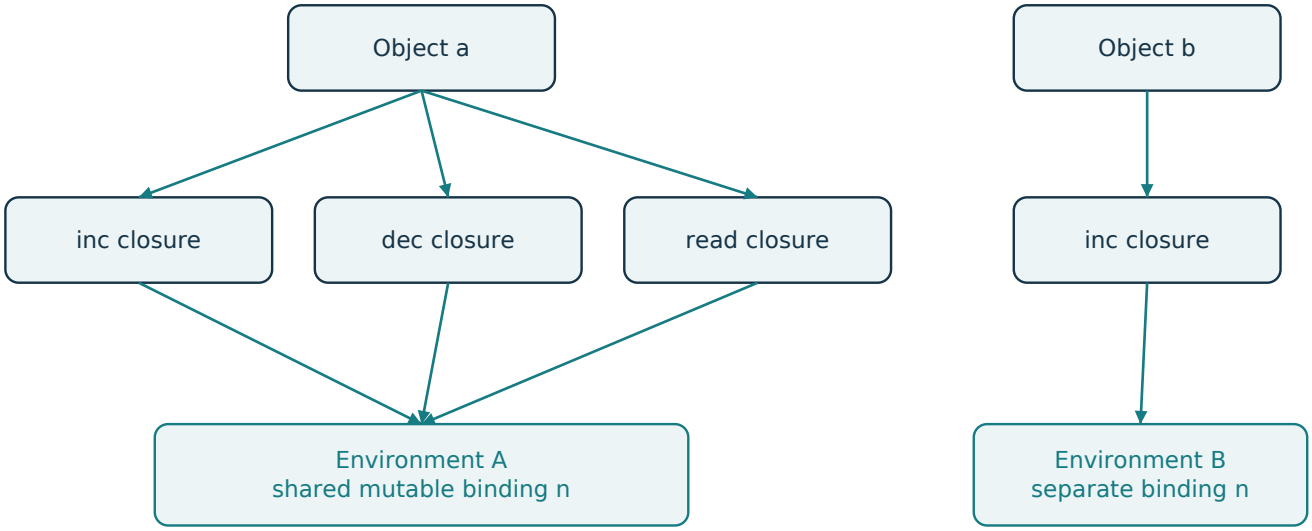


Figure 2. Three closures from one counter share a binding. A separate factory invocation creates an independent environment; the environment can outlive its creator's native frame.

The implementation maps each part to existing Hermes operations. `CreateScopeInst` calls `shllvm_env_create`. Variable indices assigned after lowering identify the environment slots. `StoreFrameInst` writes a slot through `shllvm_env_store`; `LoadFrameInst` reads it through `shllvm_env_load`. The store uses Hermes's write barrier, allowing the collector to track newly stored heap references. [1] [2] [5]

`CreateFunctionInst` calls `shllvm_closure`, which creates a real Hermes function object containing the generated native function pointer, metadata, compilation unit, and environment. At invocation, the callee recovers its environment from the current closure. Scope-resolution helpers walk parent environments for captures from more distant lexical scopes.

The creator's native frame can disappear while the environment survives. In the example, the returned object retains the closures, and those closures retain Environment A. That reachability, not the stack frame of `counter`, keeps `n` alive. The absent-environment case is handled explicitly as undefined/null helper input; the bridge must not fabricate an object-tagged null address and present it to the collector.

Nested arrows such as `a => b => c => a + b + c` extend this chain. Each invocation can introduce a new environment, and the innermost closure reaches the required outer bindings. Recursive closures add another useful case:

```
function recursiveClosure() {
  let f = n => n === 0 ? 0 : n + f(n - 1);
  return f;
}
print(recursiveClosure()(20)); // Expected mathematically: 210.
```

Here the captured `f` slot refers to a closure that itself refers to the environment containing `f`. A tracing collector can handle this cycle. The compiler must preserve shared-cell access rather than substitute an incomplete snapshot during closure creation.

There are two lifetimes to distinguish. Heap reachability controls closure/environment lifetime. The native code mapping and `SHUnit` metadata must also remain available whenever a closure can execute.

Upstream specifies that a unit, once associated with a runtime, persists until that runtime is destroyed. Unloading code while a surviving closure points to it violates a different contract that garbage collection alone cannot repair. [5]

2.8. Objects, properties, arrays, and observable order

Objects are allocated by Hermes. The emitter uses helpers for named or computed reads, assignment, defining own properties, accessors, and deletion. Named operations use interned symbol IDs and per-string property caches; computed operations pass a rooted key value. A cache accelerates the runtime's lookup protocol without replacing its semantic checks. [1] [2]

Assignment and definition are distinct. Assigning a property may invoke an inherited setter; defining an own property creates or changes a property on the object itself. Strict and non-strict writes also differ in failure behavior. The emitter chooses corresponding helper variants, including operations with an explicit receiver. The `try_flag` on global-property helpers concerns errors for missing global bindings, not JavaScript `try` syntax.

Object literals lowered into buffers are expanded into ordinary allocation and property definitions in this first backend. Where the lowered IR contains named fixed-property operations, the emitter preserves the property name instead of assuming the optimized C backend's storage offsets. Explicit prototypes use the runtime's parent-aware allocation helper and a registered empty-shape cache. Typed object allocation and private fixed-slot properties are rejected rather than silently translated with ordinary object semantics.

Arrays require a subtle distinction:

```
const hole = [,];
const present = [undefined];
print(0 in hole, 0 in present); // Expected: false true.
```

Both arrays have length one, but only the second owns an element at index zero. The emitter allocates the logical array length and skips `LiteralEmpty` entries. It still defines an element whose value is undefined. Nonliteral element stores already introduced by frontend lowering remain in their IR order. This preserves the frontend's evaluation sequence rather than recomputing expressions during allocation. [1] [2] [7]

These are also memory-management concerns. A getter, setter, conversion, or array callback can reenter generated JavaScript. Root frames, environments, outgoing arguments, and code mappings must remain valid across that nested execution.

2.9. Dynamic arithmetic and direct floating-point operations

The backend has two arithmetic paths. General JavaScript operations call Hermes helpers, which preserve coercion and exception behavior. An addition with an unknown operand type cannot simply become LLVM `fadd`: it might concatenate strings or run user code during conversion. [1] [2]

When Hermes IR already contains a floating operation, the emitter loads its known numeric operands, bitcasts their value bits to `double`, and emits `fadd`, `fsub`, `fmul`, `fdiv`, `frem`, or `fneg`. Results pass through `shllvm_number`, which uses the runtime's untrusted-double constructor before they become tagged values again.

The emitter attaches **no fast-math flags**. JavaScript exposes signed zero, infinities, and NaN-related behavior, so transformations requiring assumptions about their absence would need a stronger justification. Floating comparisons use ordered predicates for equality and ordering, and unordered-or-not-equal for inequality: `oeq`, `olt`, `ole`, `ogt`, `oge`, and `une`. This gives the intended NaN behavior for the typed comparison operations. [1] [9]

For example, a compiler must preserve the distinction between `1 / 0` and `1 / -0`, and must not turn `NaN !== NaN` into false. The root-slot representation does not itself enforce these rules; correct operation selection and the runtime's numeric constructor do. Likewise, bitwise JavaScript operations retain their runtime conversion rules rather than applying integer instructions directly to arbitrary tagged values.

2.10. Exceptions and a function-local nonlocal return

The current implementation reuses Hermes's setjmp/longjmp exception convention. It does not introduce a separate LLVM `invoke`/landing-pad exception model. A function containing lexical try regions installs one native `SHJmpBuf` and uses a small state variable to identify the active region. [1] [2] [5]

The setup is deliberate:

1. Ask the bridge for the target's `sizeof(SHJmpBuf)` and allocate aligned storage.
2. Initialize a volatile `try_state` to zero, meaning no active lexical handler.
3. Call `shllvm_try_begin` to link the buffer and obtain its native jump-buffer address.
4. Call `_setjmp` **inside the generated function**, with `returns_twice` on its LLVM declaration and call.
5. Continue normally for zero; dispatch a caught exception for a nonzero result.

Placing setjmp in a helper that returns would be incorrect: a later longjmp would attempt to resume an activation that has already ended. The bridge prepares the buffer, but the generated function owns the continuation point. LLVM's `returns_twice` attribute informs optimization about this unusual return behavior. [9]

`TryStartInst` writes its nonzero region ID. `TryEndInst` restores the enclosing region or zero. A `CatchInst` similarly establishes the enclosing handler and retrieves/clears the pending exception. Source-level finally behavior has already been expressed by the frontend's control flow; the emitter follows that graph, including paths through return, break, and continue.

On exceptional return, the dispatcher reads `try_state` and calls `shllvm_catch_prepare` with the function's **root-frame storage**, incoming frame, and outgoing argument capacity. This delegates to `_sh_catch_no_pop`, restoring runtime root/register-stack state after intervening calls have unwound. A switch then chooses the matching catch edge, including any required phi transfers. State zero follows a pop-and-rethrow path to an enclosing function's handler. [1] [2]

The emitter makes its ordinary raw value loads and stores volatile in functions containing try regions, including phi stores. Together with memory-resident state and `returns_twice`, this conservatively preserves mutations needed after a nonlocal return. Volatile is not a collector, a synchronization mechanism, or an independent proof of exception correctness.

Exception tests should examine more than a thrown integer. An exception object can retain a lambda; a finally block can replace a pending return; a nested handler can rethrow after updating a captured variable. The `exceptions.js` fixture illustrates these interactions. They exercise the relationship between language control flow, native continuation state, and heap reachability rather than just one branch instruction.

2.11. Compilation units and stable descriptors

Generated code needs data as well as function bodies. `ModuleEmitter` produces ASCII and UTF-16 pools, string records containing offset/length/hash, a unit-index global, and function descriptors. UTF-16 conversion preserves surrogate code units, and the high offset bit distinguishes the non-ASCII pool. [1] [5]

Function descriptors use an intentionally simple boundary format:

```
%SHLLVMFunctionInfo = type { i32, i32, i32, i32 }
```

The fields hold the name index, expected explicit argument count, function kind, and invocation prohibition. Native `SHNativeFuncInfo` contains C bitfields. The bridge copies descriptor fields into that structure using compiled C assignments; the LLVM emitter never guesses the target compiler's bitfield layout. [2] [3]

`sh_export_<unitName>` constructs the unit, which owns runtime symbol/cache arrays and native function metadata. Generated string pools and code pointers remain referenced by that unit. When requested, a generated `main` calls `shllvm_run`, which checks the runtime model, initializes Hermes and console bindings, initializes the unit, runs the event loop, and performs cleanup. This provides a host executable lifecycle, not a device scheduler or hot-reload manager.

2.12. Reading and running the generated program

The following commands assume a configured `build-llvm` directory with `shermes` built, matching LLVM tools available, and execution from the repository root. Set `LLVM_BIN` to your LLVM installation's `bin` directory if those tools are not otherwise discoverable. The commands are reproducible exercises; their inclusion is not a report of a particular validation run. [8]

```
# Preserve easily inspected IR: minimize both optimization stages.
python3 utils/shllvm.py emit --build-dir build-llvm \
  --hermes-opt O0 --llvm-opt 0 \
  -o build-llvm/lesson/closures.ll test/LLVMNative/closures.js

# Compile, link, and execute a native program.
python3 utils/shllvm.py run --build-dir build-llvm \
  -o build-llvm/lesson/closures test/LLVMNative/closures.js

# Execute the verified bitcode through host LLVM lli.
python3 utils/shllvm.py jit --build-dir build-llvm \
  -o build-llvm/lesson/closures-jit test/LLVMNative/closures.js
```

The driver enables ES6 block scoping and temporal-dead-zone checks, emits `.ll`, assembles it, verifies it, and applies the selected LLVM optimization pipeline with verification. It publishes the readable IR and optimized `.bc` after successful processing. Native builds link the static bridge archive plus Hermes libraries; JIT execution loads the shared bridge into `lli`. Neither path regenerates C for the guest program. The native executable still depends on its runtime libraries and build configuration.

Direct `shermes -emit-llvm` invocation also enables block scoping and TDZ checks by default, while respecting explicit `-Xes6-block-scoping=false` and `-Xenable-tdz=false` overrides. The existing C-emission oracle needs the corresponding enabled flags for comparison. These checks retain the pinned upstream frontend's language restrictions: the supplied reassignment-to-`const` example is rejected during compilation, and a statically provable read before lexical initialization can also be rejected even inside `try`. Standard JavaScript can instead expose catchable runtime errors in those examples. Closure-mediated runtime TDZ checks are exercised separately; enabling TDZ does not establish complete ECMAScript acceptance. The distinction is recorded in the lexical-default checks and the differential results.

The adjacent `.build.json` records source/IR hashes, the bridge hash where available, commands, optimization choices, sanitizer configuration, and `program_c_generation: false`. This helps connect a result to its actual inputs. The external `lli` path is host execution, not an embedded ORC integration or an RV32 emulator. A subprocess timeout limits a run from outside; it is not an instruction-fuel mechanism within generated code. [8] [10] [11]

2.13. Connecting the host backend to PicoCalc

The PicoCalc integer-world lane has a fixed state structure, a capability-call ABI, and an RV32 instruction interpreter. Its direct emitter can preserve that contract without introducing a dynamic heap. The Hermes lane introduces managed objects, closure environments, runtime services, and native code lifetimes. The integration chapter documents these separate contracts in detail. [12]

A defensible initial dynamic-world design gives each generation its own runtime. The host prepares a candidate, serializes explicitly supported application state into it, checks migration, and switches event delivery only after success. Old code remains mapped until no invocation or retained callback can reach it. Raw Hermes values, environment addresses, and native function pointers are unsuitable as a cross-runtime state format.

Several device tasks remain substantive engineering work: establish the RV32 value/callback ABI, port required allocation and exception services, provide double-precision support appropriate to the target, validate collector roots and barriers, manage executable memory/cache synchronization, and connect capability effects to the world's transaction protocol. Removing the bridge's 64-bit assertion accomplishes none of those tasks.

Heap rollback and scheduling also need explicit policies. Canceling display effects does not undo mutations already made to a Hermes object. A GC safepoint is not automatically a fuel check. LLVM's ability to emit RISC-V instructions therefore supplies one compilation component, while runtime portability and world supervision remain separate obligations. [12] [13]

2.14. Laboratory exercises and evidence

Use the exercises below to connect the implementation to observable behavior. Record tool versions, command lines, hashes, expected output, actual output, and any failure before drawing conclusions.

1. **Shared and independent captures.** Extend `counter` with a reset method. Predict results for two counters and several retained methods. Inspect emitted calls to environment load/store helpers, then compare native and JIT behavior.
2. **Parallel phi transfers.** Write a loop that rotates three values. Find its edge blocks in minimally optimized IR. Explain which inputs would be lost by interleaving loads and stores. Compare multiple optimization levels.
3. **Property effects and holes.** Combine a getter that logs access with an array containing a hole and an explicit undefined. Predict both output order and `in` results before running it.
4. **Exceptions and captures.** Throw an object containing a closure; update its captured variable in finally; call it in an outer catch. Trace handler-state changes and explain which roots retain the environment.
5. **An unsupported construct.** Compile an existing negative fixture and verify that it fails with a diagnostic. Preserve an old output file and check that failed emission does not replace it with partial IR.

Each technique answers a different question. LLVM verification checks IR structure. Differential execution against an independent JavaScript implementation checks selected observable semantics. Collector stress checks root/lifetime assumptions. Sanitizers can expose native memory mistakes. None alone proves complete language conformance, and passing a host test does not demonstrate device execution. Future optimizations should preserve these distinctions while reducing root traffic, metadata overhead, or redundant runtime calls.

3. Validation, findings, and boundaries

3.1 What each validation layer establishes

LLVM assembly and verification check the representation: types, instructions, blocks, and control flow must form a valid module. They cannot determine whether a getter ran in the right order or two closures share the right binding. The native differential suite therefore compares the program's actual output with both Node and the unchanged Static Hermes C backend. Node provides an independent language implementation; the C backend holds the Hermes frontend and runtime largely constant while changing the code generator.

The comparison controls matter. Both Hermes paths use ES6 block scoping and temporal-dead-zone checks. Without matching configuration, a difference may be a frontend-mode difference rather than a backend error. Tests use deterministic output; collection is explicitly requested where it is part of the test. A timeout limits the external test process, and does not claim that native guest execution has instruction fuel.

Evidence layer	Recorded final outcome	Scope
Host source cases	22 / 22 passed	16 positive, 3 runtime-error, 3 compile-rejection
Native differential executions	64 / 64 comparisons passed	Hermes O0 and O, each at LLVM O0 and O2; compared with Node and C
LLVM JIT executions	3 / 3 comparisons passed	Functions, mutable closures and forced collection
Runtime-error executions	6 / 6 expected errors	Three sources at both Hermes frontend settings
Compiler publication checks	6 / 6 passed	Default/stdout/exported output, C bypass and preservation on failure
Raw lexical-default checks	5 / 5 passed	Default behavior, explicit overrides and static TDZ diagnostic
RV32 integer-world regression	24 / 24 passed; no skips	Complete recorded socket-enabled simulator/service run
RV32 replay artifacts	6 verified modules; 11 trace records	Linked ELF images, actual instruction execution and reload trace

The main suite counts source cases separately from executions. Sixteen positive sources, two Hermes optimization settings, and two LLVM optimization settings produce 64 native comparisons. Three additional source cases exercise runtime errors at both Hermes settings, producing six executions. Three source cases are expected compile rejections. The three `lli` comparisons are another execution path for selected positive programs, not three more source cases.

The counter example is a separate successful native/JIT demonstration. It is not silently added to the suite's 64-comparison or three-JIT counts. The driver and lexical-default suites also have their own denominators.

3.2 Functions, shared capture, and collection

The function suite exercises recursive factorial, mutual recursion, higher-order calls and an escaping returned closure. Its recorded output is:

```
40320 true true 42 42
```

The closure suite checks shared mutable state, separate factory invocations, nested captures and recursion through a captured binding:

```
41 42 41 41 101 41
42 43
210
```

The allocation fixture creates 50,000 temporary object/array structures, retains independently created closures, confirms the `gc` hook is a function, calls it unconditionally, and reads retained state afterward:

```
gc-hook function
2 alive after collection 0,5000,10000,15000,20000,25000,30000,35000,40000,45000
```

This is materially stronger than merely allocating a few objects and hoping collection occurred. In the pinned runtime the hook calls `runtime.collect("forced")`. Nevertheless, this experiment does not exercise every collector schedule, every allocation-failure path, or every possible object graph. Its specific result is that the tested live closure graph survives the explicit collection request and remains callable.

3.3 Errors, language defaults, and publication

The runtime-error fixtures check that constructing an arrow and calling a nonfunction raise TypeError, and reading a missing global raises ReferenceError. The exception fixture also checks nested handlers and finally paths, including values retained through throwing and catching. The raw CLI checks establish default `let` loop captures of `0 1 2`; explicitly disabling block scoping produces the inherited legacy behavior `3 3 3`.

A closure-mediated read of an uninitialized lexical binding raises a catchable ReferenceError with default direct-emission flags. Explicitly disabling the TDZ option instead yields undefined. An obvious same-function TDZ violation is rejected statically by the upstream frontend. Likewise, the pinned frontend rejects assignment to a `const` binding during compilation, even when it appears inside `try`. Standard JavaScript can make such assignment a catchable runtime TypeError. These retained frontend boundaries are documented rather than presented as language conformance.

Failure publication was checked by leaving a known old output file in place and compiling invalid or unsupported input. The driver reports failure, retains that file, and emits no partial stdout module. Emission with a deliberately nonexistent `CC` still succeeds, establishing that the direct compiler's output stage does not invoke a C compiler for the guest program.

3.4 Integration bugs found during the work

The diary retains failed attempts because they explain why the final design contains its less obvious safeguards.

Finding	Correction	Why it matters
Loop callbacks observed one final value	Enabled ES6 block scoping and TDZ by default in direct mode; matched flags in the C reference	Language configuration changes closure semantics before code generation
Error construction crashed in stack-trace recording	Set `SHLocals.src_location_idx` to the runtime's zero invalid-location sentinel	Native metadata is part of correctness, even when normal arithmetic works
Shared bridge symbols were hidden	Exported bridge entry points in the CMake object target	Native linkage and JIT lookup need visible ABI symbols

Finding	Correction	Why it matters
Uninit was treated like Empty	Matched upstream's undefined encoding and preserved TDZ masks	Ordinary undefined must not be confused with an uninitialized lexical binding
Explicit object prototypes lacked runtime cache state	Added a registered weak empty-shape cache to the unit	Class and inherited-object allocation must use the actual runtime contract
Root admission was too permissive	Limited combined root/argument slots to 1,024	A source function must not request an enormous explicit native frame
Specialized object instructions could change semantics	Rejected unsupported typed forms before ordinary-object lowering	A valid-looking substitute can silently change property attributes

The native frame limit bounds only its explicit tagged payload to approximately 8 KiB. Native headers, jump buffers, compiler spills, recursion and runtime frames add further use. Both the native stack checks and the admission limit are useful safeguards; neither is a complete resource budget.

3.5 Sanitizer and platform qualifications

The compiler, Hermes runtime and bridge were built with AddressSanitizer. Generated LLVM function bodies are not explicitly ASan-instrumented, so the result must not be described as full generated-code sanitizer coverage. The execution environment blocked LeakSanitizer's process inspection; the recorded commands set `ASAN_OPTIONS=detect_leaks=0`. Address checking remained active. This setting is an environment qualification, not a recommended default for every user.

The validated dynamic backend is Linux x86-64 with LLVM/Clang 18.1.3 and a 64-bit Hermes runtime. The bridge asserts eight-byte pointers and values. A separate Clang ABI probe confirmed that the native callback returning `SHLegacyValue` uses the `i64 (ptr)` LLVM signature on this host. That experiment does not establish an ABI for RV32 or another operating system.

3.6 What this release does not establish

The supported ordinary function, object, closure and class cases are substantial, but the emitter does not cover every Hermes instruction or every ECMAScript construct. Generators and direct eval are explicit negative fixtures; regular-expression literal creation is also rejected by the driver tests. BigInt literal lowering, specialized typed objects/fast arrays and private-field operations remain outside the implemented set. No Test262 pass, full upstream suite pass, or exhaustive class-feature coverage is claimed.

The host JIT path uses an external `lli` process. It is a real LLVM JIT execution path, but it is not a new embedded ORC API, a native-code sandbox, or a hot-reload manager. Compilation units, code and constant storage must remain alive for the runtime's lifetime. The host backend has no automatic rollback of arbitrary heap mutation and no world capability boundary. Those concerns are addressed as integration work in Chapter 4.

The PicoCalc lane's final 24-test run establishes the inherited bounded integer-world behavior in the simulator. It includes 200 seeded C-versus-LLVM event comparisons and the original 160 Node oracle comparisons. Six modules were independently assembled and verified, linked to RV32IM ELF images, and executed in the instruction runner. These counts are not host JavaScript conformance tests and do not establish physical-device execution.

3.7 Reading the evidence

The final host results are `docs/llvm/evidence/final-validation.json` and `final-validation.log`. The original `validation.json` and `validation.log` remain as historical evidence. The command-publication and lexical-default records are `driver-validation.json` and `lexical-defaults-validation.json`. `final-validated-files.json` connects the final execution evidence to hashes of the actual inputs and binaries; `validated-files.json` preserves the earlier capture. Selected emitted IR, disassembly and build metadata are in `evidence/samples/`. Toolchain package provenance and the native callback ABI probe are in `evidence/toolchain/`.

The device results are `picocalc-llvm-poc/logs/direct-llvm-validation.json` and `direct-llvm-tests-with-sockets-final.txt`, together with the replay trace and ELF audit. Earlier failures and incomplete attempts are explicitly named and retained for the diary; they are not the final gate. The release manifest hashes the packaged files, and the patch verification records an application to the exact upstream base with an identical resulting source tree.

3.8 Post-review diagnostic parity

Final review identified one diagnostic parity gap: direct emission did not consult the requested lowered-Hermes-IR verification option. The emitter now follows the existing C backend's verifier sequence before assigning environment indices. The original full-matrix records and hashes are preserved. The separate post-review record identifies the final source/compiler hashes: 32 enabled/disabled emission pairs were byte-identical, all 64 modules assembled and passed LLVM verification, and both counter executions (native and JIT) matched the prior output. That focused guard check initially left the broad native matrix unchanged. The pinned Release build compiles the upstream Hermes verifier as a no-op unless HERMES_SLOW_DEBUG is enabled; these checks therefore establish flag plumbing and output invariance, not a malformed-Hermes-IR rejection experiment. LLVM assembly and LLVM verification remain real checks in this build.

Final reproduction review also found that the wrapper's automatic CMake command refreshed only the shared bridge, while native linking uses the static archive. It now requests both bridge targets. Separate native and JIT counter runs with automatic building enabled verified the dependency correction and matched the prior output. This driver-only change is recorded separately from the full differential matrix and does not change the emitter's instruction translation.

The subsequent automatic dependency build also refreshed the Hermes VM binary. To validate that concrete final artifact, the complete host matrix was then repeated with the final compiler, driver, runtime and bridges: all 22 cases passed, including 64 native comparisons, three JIT comparisons and six checked runtime-error executions. No quick mode or JIT skip was used. The final matrix is the release gate; earlier and focused records remain available to explain how the implementation reached it.

4. PicoCalc integration and the device boundary

This change provides a direct LLVM backend at two different layers. Keeping their evidence separate is essential to understanding what now works and what a physical-device port still requires.

Property	PicoCalc integer world lane	Static Hermes dynamic language lane
Frontend	Existing restricted TypeScript-parsed JS/i32 grammar	Actual Static Hermes parser and IR pipeline
New backend	`compiler/js2llvm.js`	`lib/BCGen/SH/LLVMNative/LLVMBackend.cpp`
Per-program output	Textual LLVM IR, then a prelinked ELF32 module	Textual LLVM IR calling the Static Hermes runtime bridge
Runtime values	Fixed `PicoState` and i32 temporaries	Hermes values, objects, closure environments, native function objects
Target evidence	RV32IM instructions executed in the existing simulator	Linux x86-64 native execution and host `lli`; see the final host validation records
Managed heap and collector	None	Actual host Static Hermes runtime owns the heap and collector
Reload behavior exercised	State-preserving generation replacement, migration rollback, stale-event rejection	Code/runtime lifetime must be retained as a generation; device integration is not implemented
Physical ESP32-P4 execution	Not tested	Not tested

The integer lane's **24-test complete regression run** is recorded in `picocalc-llvm-poc/logs/direct-llvm-tests-with-sockets-final.txt`. Its source hashes and counts are in `logs/direct-llvm-validation.json`. The collector `scripts/collect_direct_llvm.py` assembled and verified six directly emitted modules, linked RV32IM ELF images, and ran an eleven-record reload scenario. That scenario retained score 3 across a successful migration, reached score 43 under the new implementation, rejected a failing replacement, and then reached score 129 using the still-active generation. The final epoch was 2.

These results support preservation of the original world protocol for the exercised cases after removing the per-module C stage. They do not establish that objects or closures run in the RV32 interpreter or on physical PicoCalc hardware.

The host backend reuses the full dynamic Hermes runtime, including its heap, collector, objects, and closure machinery, through a selected set of emitted instructions. This does not mean complete ECMAScript acceptance. Generators, direct `eval`, and regular-expression literals are rejected; typed-object and private fixed-slot operations are outside this backend. The pinned frontend also rejects the supplied `const` reassignment and some statically provable TDZ violations during compilation, even inside `try`. Runtime TDZ behavior through captured bindings is tested separately. See `evidence/validation.json`, `evidence/driver-validation.json`, and `evidence/lexical-defaults-validation.json`.

4.1. The shared architectural idea

Both lanes place the compiler on the host. A module's source becomes LLVM IR, then native machine code. The integer lane's world supervisor owns execution policy and capability access. Connecting the dynamic host runtime to that supervisor remains integration work. LLVM is not the operating system, JavaScript runtime, garbage collector, or state migration mechanism.

In the integer lane, the LLVM IR has no external runtime calls. Capability requests use a RISC-V `ecall`, and the simulator enforces memory access and instruction fuel. The ABI is a 16-byte `PicoState`, an 8 KiB stack, and one of two prelinked code slots, `0x100000` and `0x200000`.

In the dynamic lane, generated IR calls the bridge in `runtime.c`, which in turn uses Static Hermes's existing value operations, object model, closure creation, environment stores, function call machinery, and garbage-collector roots. The bridge is compiled once for its runtime configuration. Guest program bodies do not pass through generated C.

4.2. Why a closure changes the reload problem

A closure combines executable code with an environment containing captured bindings. An object can retain a closure, and that closure can retain further objects. Replacing a code pointer does not transfer this graph to another runtime or make the old graph safe to collect.

The upstream `SHUnit` contract says that a unit, once associated with an `SHRuntime`, persists until that runtime is destroyed. The generated native function objects and their metadata refer to that unit. Unmapping an old generation's machine code while a surviving closure can call it would violate the lifetime contract.

For PicoCalc, the defensible first integration is therefore **one runtime per world generation**:

1. Compile and validate a candidate module on the host.
2. Create the candidate runtime and instantiate the module without exposing live external effects.
3. Migrate a deliberately serialized state representation. Do not copy native pointers, Hermes values, object addresses, or closure environments between runtimes.
4. Check schema and resource policy; activate the candidate only after success.
5. Direct new events to the candidate generation and reject stale events using the existing epoch rule.
6. Release the old runtime, all old handles, and finally its code mapping after no invocation or externally held callback can reach it.

This sequence describes the required dynamic-world integration. The current integer simulator already exercises its simpler fixed-state version. It does not yet instantiate a Hermes runtime inside a world.

Plain JSON state may be sufficient for a first application, but it has its own boundary: functions, object identity, cycles, undefined-valued properties, BigInts, and non-finite numbers need an explicit policy. A closure is behavior to recreate in the new generation, not a raw address to migrate.

4.3. Transactional effects require host bindings

The existing simulator copies fixed state and stages panel/log effects until the native turn completes. A dynamic runtime needs corresponding bindings that append effect proposals to a generation-owned transaction. Direct access to host file descriptors, panel drivers, or shared mutable objects would bypass that protocol.

The host also needs to decide whether heap mutation itself rolls back. A Hermes object modified during a failed call does not automatically return to its prior value merely because effect delivery was

canceled. Practical first choices include application-level pure reducers over serialized state, a fresh runtime for each speculative turn, or a separate snapshot/transaction design. The present compiler change does not implement any of these dynamic heap rollback strategies.

Likewise, the integer interpreter's instruction counter cannot simply be reused for arbitrary native Hermes code. A device port needs a defined interruption mechanism: explicit compiler-inserted polling, protected execution with a timer and reliable unwind policy, or another independently validated scheduling design. Garbage-collector safepoints and world fuel checks are different mechanisms with different guarantees.

4.4. What an RV32 Hermes port entails

The bridge currently contains a deliberate `sizeof(void *) == 8` assertion. It prevents the tested host callback ABI from being advertised as an RV32 runtime ABI. Deleting that assertion is not a port.

The host emitter also admits at most 1,024 combined root/argument slots, including scratch/minimum argument storage. That caps explicit tagged payload at 8 KiB; it does not cap total native stack consumption. Root headers, exception buffers, compiler spills, runtime frames, and recursion add storage. This admission rule is distinct from the integer simulator's actual 8 KiB guest-stack region.

The port has to establish at least these contracts:

- **Value and pointer model.** Check the selected Hermes VM model, tagged value representation, compressed pointers if enabled, heap address limits, and collector support for the intended target.
- **Calling convention.** Prove that the generated `i64` native function return and all bridge prototypes agree with the target C ABI. Aggregate returns, register allocation, alignment, and pointer widths require target validation, not host extrapolation.
- **Runtime services.** Supply allocation, stack bounds, platform atomics, exception machinery, standard-library dependencies, and any threading or timer services required by the selected runtime configuration.
- **Floating-point support.** JavaScript uses binary64 Numbers. RV32IM code requires suitable software helpers; a single-precision F extension alone does not provide double-precision execution. The actual hardware/toolchain configuration and runtime libraries must be tested together.
- **Collector and root correctness.** Rebuild the bridge against the exact target runtime. Verify root-frame alignment, write barriers, moved-object updates, allocation failure, and exception unwinding under collection stress on that target.
- **Executable code management.** Define executable memory placement, instruction-cache synchronization, loading/relocation, and generation lifetime. The prototype's two fixed synthetic slots are a test convention, not a physical-device memory map.
- **OS policy.** Connect capabilities, turn scheduling, input queues, display effects, memory budgets, state migration, and failure reporting to the actual PicoCalc nucleus. The current simulator does not implement the physical panel, FreeRTOS, USB, or hardware privilege boundaries.

Generating an RV32 object file can be a useful intermediate check, but it does not demonstrate that its imports link, its collector works, or the result fits the device. Conversely, host execution of object/closure tests does demonstrate the new direct backend's behavior against the actual Hermes runtime, within the recorded feature and test scope.

Source-reading pointers for these contracts are collected in Chapter 6. The standalone integration chapter retains its own local reference list.

5. Build, inspect, execute, and reproduce

The driver in `utils/shllvm.py` accepts JavaScript, invokes Static Hermes' new `-emit-llvm` mode, assembles and verifies the resulting LLVM IR, and uses LLVM to produce an executable or run the module through `lli`. The shared/static bridge is built once through CMake dependency tracking. Individual JavaScript programs do not produce C source and do not invoke a C frontend.

5.1. Build prerequisites

Use a native Linux x86-64 host, a C++ build toolchain, CMake, Ninja, Python 3, and LLVM 18 tools (`clang`, `llvm-as`, `opt`, `lli`). The recorded toolchain was LLVM 18.1.3. The runtime bridge currently checks for eight-byte host pointers; the presence of other targets in LLVM does not establish a working device runtime.

A representative configuration is:

```
cmake -S . -B build-llvm -G Ninja \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_C_COMPILER=clang \
  -DCMAKE_CXX_COMPILER=clang++ \
  -DHERMES_ENABLE_ADDRESS_SANITIZER=ON
cmake --build build-llvm --target shermes shllvm_runtime shllvm_runtime_shared --parallel 3
```

The recorded build used `-O1 -DNDEBUG` for its C++ release flags to reduce build cost; this is distinct from the frontend and LLVM optimization choices for each JavaScript program. Set `LLVM_BIN` if the LLVM executables are not on PATH, and `HERMES_BUILD_DIR` for a different configured build directory. Both also have corresponding command-line options.

5.2. Commands

```
# Keep human-readable LLVM IR and optimized bitcode.
python3 utils/shllvm.py emit test/LLVMNative/closures.js -o out/closures.ll

# Produce a native executable, along with .ll, .bc and .build.json artifacts.
python3 utils/shllvm.py build test/LLVMNative/closures.js -o out/closures
./out/closures

# Compile and execute with a subprocess timeout.
python3 utils/shllvm.py run test/LLVMNative/classes.js

# Real LLVM JIT execution in the lli host process.
python3 utils/shllvm.py jit test/LLVMNative/gc_closures.js

# Select frontend and LLVM optimization independently.
python3 utils/shllvm.py run --hermes-opt O0 --llvm-opt 0 \
  test/LLVMNative/exceptions.js
```

The driver enables ES6 block scoping and temporal-dead-zone checks explicitly. These options matter for a closure created inside a `let` loop and for a closure that reads a lexical binding before its declaration executes. The original C backend requires the same options when used as a semantic comparison.

Arguments after `--` are forwarded to the generated program/`lli` program. Static Hermes itself uses `--` to distinguish runtime flags from script arguments, so an invocation that needs script arguments can contain a second separator. `--no-runtime-build` uses an already-built matching bridge and is intended for validation batches. Normal invocations ask CMake to ensure the bridge and dependent libraries are current.

A failed frontend emission or LLVM verification leaves an existing LLVM output untouched. Executable publication also uses a temporary file followed by a rename. Build metadata contains source, IR, compiler, runtime and executable hashes and the commands that were run.

5.3. Native execution and JIT are separate paths

`build` and `run` link compiled LLVM code with a once-built static bridge and the Hermes runtime/console shared libraries. `jit` asks LLVM's `lli` executable to JIT the bitcode and load the shared bridge. The latter is real native execution through LLVM, but it is not an in-process embedding API or a hot-reload manager. Every invocation currently creates its own process/runtime lifetime.

For an AddressSanitizer runtime build, the driver preloads the matching Clang ASan shared library into `lli`. The runtime and bridge are instrumented. The emitted LLVM bodies do not carry ASan instrumentation attributes, so this must not be described as complete instrumentation of all generated code.

In the execution sandbox used for this report, LeakSanitizer could not inspect `/proc`. Validation therefore set `ASAN_OPTIONS=detect_leaks=0`; address checks remained enabled. Ordinary users should only apply that setting if their environment has the same restriction. The driver does not disable leak checking automatically.

5.4. Differential validation

```
python3 utils/test_shllvm.py --jobs 2
```

Node must be on PATH. The runner executes Node with `--expose-gc`, defines `print` as `console.log`, and compares each successful program with the original C-emission backend and the direct LLVM backend. Both Hermes optimization levels (`O0`, `O`) and both LLVM levels (`0`, `2`) are exercised. The suite also executes three programs through `lli`, verifies runtime error classes, and checks that rejected constructs preserve an existing output file.

The final release run passed **22 source cases**: **16 positive programs**, **3 runtime error programs**, and **3 explicit compile rejections**. There were **64 native LLVM comparisons** with Node and the C backend, **3 successful `lli` JIT comparisons**, and **6 checked runtime-error executions**. The allocation test creates 50,000 temporary object/array structures, retains closures, confirms that `gc` exists, invokes it unconditionally, and reads the captured state successfully afterward. These are integration tests, not a full ECMAScript conformance certificate.

Final release results are in `evidence/final-validation.json` and `evidence/final-validation.log`; `evidence/final-validated-files.json` identifies the exact tested implementation, compiler, runtime and bridge. The complete suite was rerun after the final verifier guard, automatic static/shared bridge build correction and dependent VM rebuild. The original run remains unchanged in `evidence/validation.json`, `evidence/validation.log` and `evidence/validated-files.json`.

`evidence/post-review-verifier.json` records the focused verifier-flag and LLVM output checks. `evidence/post-review-driver.json` records a missing-static-archive first-run check and native/JIT executions with automatic builds enabled. `evidence/samples/` contains emitted LLVM, selected application disassembly, and build metadata for representative programs. The diary records failures found and corrected during implementation.

5.5. Explicit boundaries

Generators and dynamic `eval` are rejected by this backend. The pinned upstream frontend also rejects an assignment to a `const` binding during compilation, even inside a `try` block; standard JavaScript can instead raise a catchable runtime TypeError. That limitation is retained and tested as an explicit compile rejection. The implementation is an experimental backend over the pinned Static Hermes language/runtime pipeline, not a claim that every ECMAScript program is accepted.

The current bridge uses the existing Static Hermes unit lifecycle: a unit and its code/string storage must remain alive until its runtime is destroyed. Unloading an old code generation while closures still refer to it is not supported. Device integration, an RV32 runtime ABI, capability confinement,

transactional OS state updates and generation-based hot reload need their own implementation and validation.

5.6 Exact release layout and patch route

The complete archive has a `hermes/` source tree and a `picocalc-llvm-poc/` tree. Run host commands from `hermes/`. The archive omits build outputs, Git history, package caches and `node_modules`. It retains the relevant upstream licences, dependency sources, the PicoCalc lockfile, recorded evidence and report sources. Rebuild binaries for your installation.

For an existing clone, check out the pinned base in a separate worktree and apply `Static-Hermes-direct-LLVM.patch` with `git apply --check` followed by `git apply`. The delivered provenance records a successful independent-index application to that base and an exact resulting-tree comparison. Do not assume that a moving upstream branch accepts the patch unchanged.

To reproduce the individual compiler gates after the build:

```
python3 utils/test_shllvm_driver.py
python3 utils/test_shllvm_defaults.py
python3 utils/test_shllvm.py --jobs 2
```

For a nondefault installation, configure each test entry point explicitly:

```
python3 utils/test_shllvm_driver.py \
  --compiler /path/to/build/bin/shermes
python3 utils/test_shllvm_defaults.py \
  --build-dir /path/to/build --clang /path/to/llvm/bin/clang
python3 utils/test_shllvm.py \
  --build-dir /path/to/build --llvm-bin /path/to/llvm/bin --jobs 2
```

The main differential runner also honors `LLVM_BIN` and `HERMES_BUILD_DIR`. The raw driver check uses `--compiler`; the lexical-default check uses `--build-dir` and `--clang` (with `CLANG` supplying its default compiler). The original evidence contains absolute paths from the recording environment; these are provenance, not paths that must exist on another machine. The Python drivers resolve current tool and runtime locations.

The PicoCalc README gives its own toolchain variables, dependency installation, test commands and replay format. Its original C-oriented technical report is retained as historical context; `docs/DIRECT_LLVM.md` and the present report describe the updated compiler. The two projects' test runners are independent.

PDF regeneration uses `python3 docs/llvm/build_report.py` from the Hermes source root with ReportLab and DejaVu fonts installed. This renders the saved chapters and evidence; it does not rerun or synthesize validation results.

6. References and source-reading guide

Local links identify the implementation accompanying this chapter. Upstream links are pinned to the base commit so that the reused contracts remain reviewable. LLVM links use the LLVM 18 documentation family; consult the installed tool's version when reproducing a build.

1. Direct emitter: `LLVMBackend.cpp`. Read `FunctionEmitter::generate`, `emitInstruction`, `emitEdges`, and `ModuleEmitter::emitData` together.
2. Runtime adaptation: `runtime.c`. Follow `shllvm_enter`, `shllvm_call`, environment helpers, exception helpers, and `shllvm_create_unit`.
3. Bridge interface: `runtime.h`. Defines the fixed descriptors and helper signatures.
4. Upstream Static Hermes lowering and C emission. Compare `lowerModuleIR` with later register-specific transformations; the local file adds `lowerForLLVM`.
5. Upstream Static Hermes public runtime contract. Contains `SHUnit`, `SHLocals`, function metadata, value helpers, and exception entry points.
6. Upstream Hermes stack-frame layout. Explains the call-frame indices preserved by the bridge.
7. Upstream lowering implementation. Provides context for object/array and other instruction transformations.
8. Build and execution driver: `utils/shllvm.py`. Its commands and manifests define the reproducible workflow.
9. LLVM 18 Language Reference. Consult phi nodes, floating comparisons, fast-math flags, volatile accesses, and `returns_twice`.
10. LLVM 18 `lli` guide. Describes host bitcode execution.
11. LLVM 18 `opt` guide. Describes optimization and verification tooling.
12. PicoCalc integration and remaining port work. Separates integer-world evidence from dynamic host-runtime execution.
13. LLVM RISC-V target documentation. Describes backend target considerations rather than a JavaScript runtime port.

Additional evidence-reading references

The numbered sources above support the architecture chapter. Implementation and experiment claims also refer to the delivered source and logs named in Chapter 3. Read `test/LLVMNative/gc_closures.js` together with the upstream `lib/VM/JSLib/GlobalObject.cpp` collector hook when assessing the forced-GC test. Follow source code to its runtime implementation when an inline header declaration does not establish allocation or lifetime behavior.

For further study, use the official LLVM garbage-collection documentation to compare this explicit-root bridge with compiler-supported collector interfaces. The present implementation does not use LLVM statepoints. Use the official ECMAScript specification when distinguishing language semantics from retained Static Hermes frontend restrictions. A specification describes the required language; the test matrix describes the subset actually exercised here.